PROJECT REPORT

Automated Web Scanner: Programmed a Python Script to Audit Sites for Secure Headers and CORS Flaws

Project domain: Web Application Security
Implementation: Python command-line utility
Institution branding: Sandip University logo supplied for this report

Abstract

This project presents a lightweight Python scanner that reviews selected HTTP security headers and observes basic Cross-Origin Resource Sharing (CORS) response behavior on explicitly authorized websites. The program sends standard GET and OPTIONS requests, records status codes and response headers, evaluates a set of common security controls, and produces a JSON report. It is designed as a non-intrusive configuration review aid; findings require manual validation and are not proof that a site is exploitable.

1. Introduction

Web applications rely on browser security controls and server response headers to reduce risks such as content injection, clickjacking, MIME sniffing, information leakage, and unintended cross-origin access. Incorrect CORS rules can allow untrusted websites to read responses in contexts where browser policy permits it. A repeatable scanner can help teams identify configuration items that deserve further review.

2. Problem Statement

Manually checking security headers across multiple websites is time-consuming and can produce inconsistent notes. CORS behavior also requires careful interpretation because the presence of an Access-Control-Allow-Origin header alone does not establish a vulnerability. The project aims to automate initial observations and produce a structured report that supports security review.

3. Objectives

- Automate checks for selected HTTP security headers.
- Observe CORS-related response headers using a controlled test Origin.
- Support scanning of multiple explicitly authorized URLs.
- Generate a machine-readable JSON report and console summary.
- Explain findings and limitations to support manual verification.

4. Scope and Safety

The scanner uses ordinary GET and OPTIONS requests only. It does not crawl links, exploit endpoints, bypass access controls, brute-force, or attempt authentication. Use only against websites owned by the operator or covered by written authorization. Test environments are preferred. Some servers may reject OPTIONS or return different headers by method; these differences should be documented rather than interpreted automatically as vulnerabilities.

5. System Design

Module	Responsibility
URL validation	Accepts only complete HTTP/HTTPS URLs and rejects embedded credentials.
HTTP request layer	Sends GET and OPTIONS requests with a timeout and a descriptive user agent.

Module	Responsibility
Header analyzer	Checks HSTS (HTTPS only), CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy.
CORS analyzer	Reviews ACAO wildcard, explicit origin values, test-origin allowance, and the credentials header combination.
Report writer	Writes timestamps, response details, findings, recommendations, and limitations to JSON.

6. Methodology

1) Define the authorized target list and obtain permission. 2) Run the script with a suitable timeout. 3) Collect status codes and response headers for GET and OPTIONS. 4) Apply simple checks to selected headers and CORS observations. 5) Review the generated JSON output. 6) Manually validate relevant items against application needs, server configuration, and browser behavior. 7) Remediate through normal change control and repeat the scan.

7. Security Header Checks

Header	Purpose / review guidance
Strict-Transport-Security	Encourages HTTPS-only browser access; meaningful for HTTPS endpoints.
Content-Security-Policy	Restricts allowed content sources; deploy carefully and test for application compatibility.
X-Content-Type-Options	Use nosniff to reduce MIME-type sniffing risks.
X-Frame-Options	Can help prevent unwanted framing; CSP frame-ancestors is a modern complementary control.
Referrer-Policy	Controls referrer information shared with other sites.
Permissions-Policy	Restricts selected browser features where appropriate.

8. CORS Review

The scanner uses a controlled Origin value and records Access-Control-Allow-Origin and Access-Control-Allow-Credentials. A wildcard origin can be appropriate for public, non-credentialed resources; it is not automatically a flaw. Browsers reject wildcard ACAO for credentialed CORS requests, so a wildcard-plus-credentials response is treated as a configuration review item rather than proof of a working exploit. If a server allows the scanner's test origin, confirm whether the origin is intentionally allowlisted and whether arbitrary origins are reflected. Review allowed methods, allowed headers, credentials, cache variation, and the sensitivity of the response before drawing conclusions.

9. Execution

```
python automated_web_scanner.py https://your-authorized-site.example
python automated_web_scanner.py https://site-a.example https://site-b.example --timeout 10 --output repor
```

The output JSON includes the target, scan timestamp, GET and OPTIONS statuses, observed headers, findings, recommendations, and safety scope.

10. Expected Outcomes

- A concise inventory of selected security-header observations.
- CORS response indicators that can be reviewed by a developer or security analyst.
- A repeatable JSON report for evidence and comparison.
- Better awareness of the difference between a configuration observation and a confirmed vulnerability.

11. Limitations

The scanner does not execute browser JavaScript, test authenticated flows, inspect every endpoint, crawl a site, evaluate all security headers, or prove data exposure. Some headers may be intentionally omitted based on architecture. GET and OPTIONS responses can differ. Results depend on network conditions, redirects, caching, proxies, and the target's current configuration. Manual verification is essential.

12. Conclusion

The Automated Web Scanner provides a compact first-pass review of common HTTP security headers and selected CORS response behavior. Its standard-library implementation is easy to run and produces structured output for documentation. When used within an authorized scope and followed by manual validation, it can support secure configuration reviews without intrusive testing.

References and Further Reading

• OWASP Secure Headers Project.
• MDN Web Docs — Cross-Origin Resource Sharing (CORS).
• MDN Web Docs — HTTP response headers and Content Security Policy.
• OWASP Web Security Testing Guide — configuration and deployment management testing.